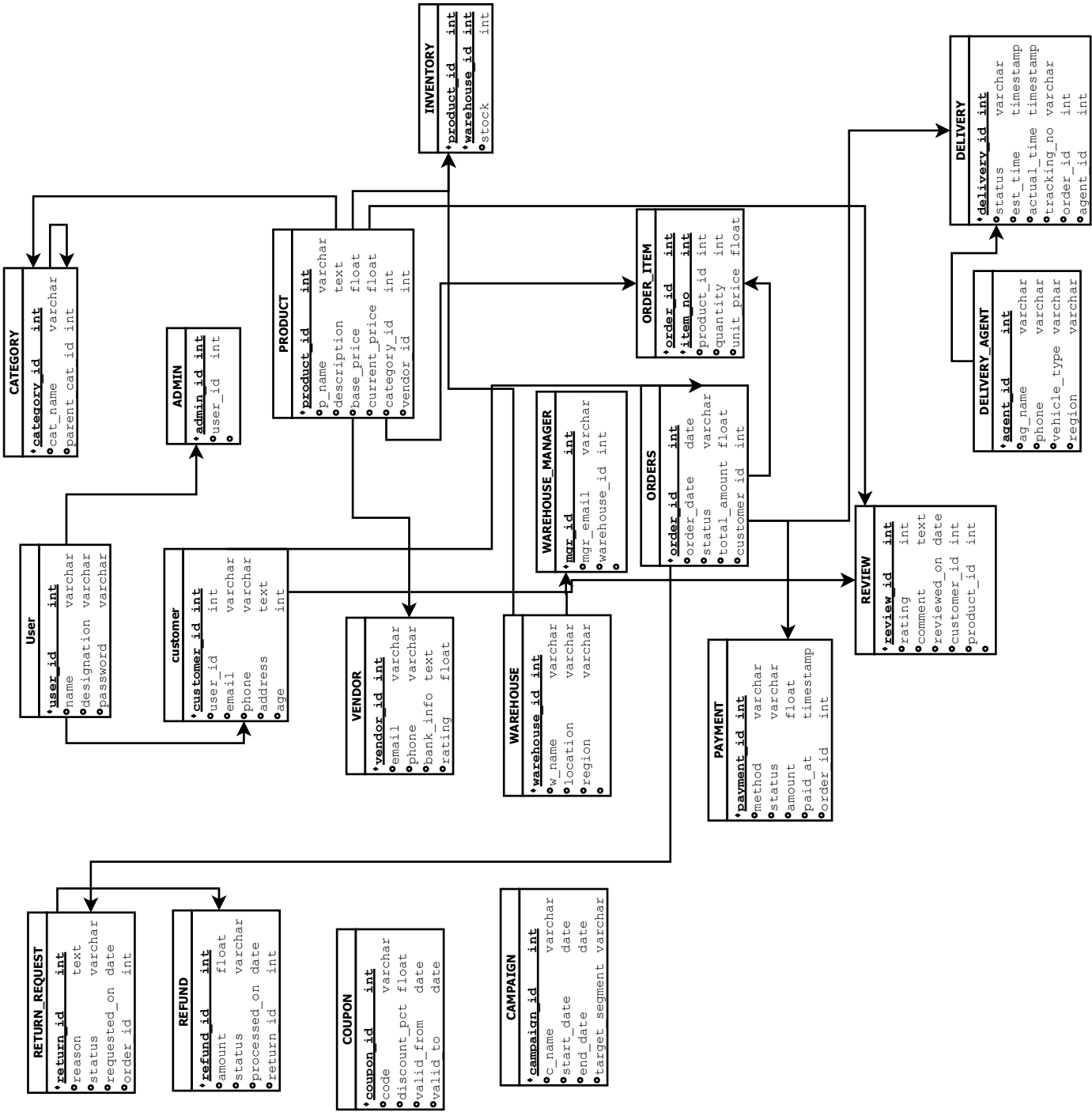




Relational Schema Analysis

Minimal Functional Dependencies | BCNF Normalization |
PostgreSQL DDL

E-Commerce Database Design

-
- 1. Minimal Functional Dependency (FD) Set
 - 2. Normalization Proof (BCNF)
 - 3. PostgreSQL DDL Scripts
-

Section 1: Minimal Functional Dependency (FD) Set

Below is the minimal (irreducible) set of functional dependencies for every relation. Each FD has a single-attribute right-hand side and no redundant determinants on the left.

USER

```
user_id → name
user_id → designation
user_id → password
```

CUSTOMER

```
customer_id → user_id
customer_id → email
customer_id → phone
customer_id → address
customer_id → age
```

ADMIN

```
admin_id → user_id
```

VENDOR

```
vendor_id → email
vendor_id → phone
vendor_id → bank_info
vendor_id → rating
```

CATEGORY

```
category_id → cat_name
category_id → parent_cat_id
```

PRODUCT

```
product_id → p_name
product_id → description
product_id → base_price
product_id → current_price
product_id → category_id
product_id → vendor_id
```

WAREHOUSE

warehouse_id → w_name
warehouse_id → location
warehouse_id → region

WAREHOUSE_MANAGER

mgr_id → mgt_email
mgr_id → warehouse_id
warehouse_id → mgr_id (each warehouse has one manager)

ORDERS

order_id → order_date
order_id → status
order_id → total_amount
order_id → customer_id

ORDER_ITEM

(order_id, item_no) → product_id
(order_id, item_no) → quantity
(order_id, item_no) → unit_price

PAYMENT

payment_id → method
payment_id → status
payment_id → amount
payment_id → paid_at
payment_id → order_id
order_id → payment_id (one payment per order)

DELIVERY

delivery_id → status
delivery_id → est_time
delivery_id → actual_time
delivery_id → tracking_no
delivery_id → order_id
delivery_id → agent_id
order_id → delivery_id (one delivery per order)
tracking_no → delivery_id

DELIVERY_AGENT

agent_id → ag_name
agent_id → phone
agent_id → vehicle_type
agent_id → region

REVIEW

review_id → rating
review_id → comment
review_id → reviewed_on
review_id → customer_id
review_id → product_id

RETURN_REQUEST

return_id → reason
return_id → status
return_id → requested_on
return_id → order_id

REFUND

refund_id → amount
refund_id → status
refund_id → processed_on
refund_id → return_id
return_id → refund_id (one refund per return)

COUPON

coupon_id → code
coupon_id → discount_pct
coupon_id → valid_from
coupon_id → valid_to
code → coupon_id (unique code)

CAMPAIGN

campaign_id → c_name
campaign_id → start_date
campaign_id → end_date
campaign_id → target_segment

INVENTORY

(product_id, warehouse_id) → stock

Section 2: Normalization Proof (BCNF)

BCNF Condition: A relation R is in Boyce-Codd Normal Form if and only if for every non-trivial functional dependency $X \rightarrow Y$ in R, X is a superkey of R. A relation must first be in 3NF before BCNF is verified. Below we confirm each table passes 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF $\rightarrow$ BCNF.

Relation	Primary / Candidate Keys	Non-Key Attributes	BCNF Justification	Result
USER	user_id	name, designation, password	All non-key attributes depend solely on user_id (PK). No partial or transitive dependencies. user_id is the only determinant $\rightarrow$ BCNF.	BCNF ✓
CUSTOMER	customer_id	user_id, email, phone, address, age	All attributes depend only on customer_id. user_id is a FK (not a determinant of other attributes here). No transitive dependency $\rightarrow$ BCNF.	BCNF ✓
ADMIN	admin_id	user_id	Single non-key attribute, trivially BCNF.	BCNF ✓
VENDOR	vendor_id	email, phone, bank_info, rating	All attributes determined solely by vendor_id $\rightarrow$ BCNF.	BCNF ✓
CATEGORY	category_id	cat_name, parent_cat_id	parent_cat_id is a self-referential FK, not a separate determinant. Only category_id $\rightarrow$ all attributes $\rightarrow$ BCNF.	BCNF ✓
PRODUCT	product_id	p_name, description, base_price, current_price, category_id, vendor_id	All non-key attributes determined by product_id only. category_id and vendor_id are FKs. No transitive dependency $\rightarrow$ BCNF.	BCNF ✓
WAREHOUSE	warehouse_id	w_name, location, region	All attributes depend only on warehouse_id $\rightarrow$ BCNF.	BCNF ✓
WAREHOUSE_MANAGER	mgr_id (also warehouse_id is a CK)	mgr_email, warehouse_id	Two candidate keys: mgr_id and warehouse_id. Every determinant (mgr_id, warehouse_id) is a superkey $\rightarrow$ BCNF.	BCNF ✓
ORDERS	order_id	order_date, status, total_amount, customer_id	All non-key attributes determined by order_id. customer_id is FK only $\rightarrow$ BCNF.	BCNF ✓
ORDER_ITEM	(order_id, item_no)	product_id, quantity, unit_price	Composite PK. All attributes fully depend on the composite key. No partial dependency (neither order_id alone nor item_no alone determines any attribute) $\rightarrow$ BCNF.	BCNF ✓
PAYMENT	payment_id (also order_id is a CK)	method, status, amount, paid_at, order_id	Two candidate keys: payment_id and order_id. Both are superkeys and the only determinants $\rightarrow$ BCNF.	BCNF ✓
DELIVERY	delivery_id (also order_id, tracking_no are CKs)	status, est_time, actual_time, tracking_no, order_id, agent_id	Three candidate keys: delivery_id, order_id, tracking_no. All are superkeys. No other determinants exist $\rightarrow$ BCNF.	BCNF ✓

Relation	Primary / Candidate Keys	Non-Key Attributes	BCNF Justification	Result
DELIVERY_AGENT	agent_id	ag_name, phone, vehicle_type, region	All attributes depend only on agent_id → BCNF.	BCNF ✓
REVIEW	review_id	rating, comment, reviewed_on, customer_id, product_id	All non-key attributes determined by review_id. FKs are not independent determinants → BCNF.	BCNF ✓
RETURN_REQUEST	return_id	reason, status, requested_on, order_id	All attributes depend only on return_id → BCNF.	BCNF ✓
REFUND	refund_id (also return_id is a CK)	amount, status, processed_on, return_id	Two candidate keys: refund_id and return_id. Both superkeys → BCNF.	BCNF ✓
COUPON	coupon_id (also code is a CK)	code, discount_pct, valid_from, valid_to	Two candidate keys: coupon_id and code. Both are superkeys → BCNF.	BCNF ✓
CAMPAIGN	campaign_id	c_name, start_date, end_date, target_segment	All attributes depend only on campaign_id → BCNF.	BCNF ✓
INVENTORY	(product_id, warehouse_id)	stock	Composite PK. Single non-key attribute. Only determinant is the composite PK which is a superkey → BCNF.	BCNF ✓

Conclusion: All 20 relations in the schema satisfy BCNF. Every non-trivial functional dependency has a superkey as its determinant. No decomposition is required.

Section 3: PostgreSQL DDL Scripts

Complete CREATE TABLE statements for all relations, in dependency order (referenced tables before referencing tables). All primary keys, foreign keys, NOT NULL constraints, and appropriate data types are included.

-- 1. USER

```
CREATE TABLE "User" (  
  user_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  designation VARCHAR(50) NOT NULL,  
  password VARCHAR(255) NOT NULL  
);
```

-- 2. CUSTOMER

```
CREATE TABLE Customer (  
  customer_id SERIAL PRIMARY KEY,  
  user_id INT NOT NULL UNIQUE REFERENCES "User"(user_id) ON DELETE CASCADE,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  phone VARCHAR(20),  
  address TEXT,  
  age INT CHECK (age >= 0)  
);
```

-- 3. ADMIN

```
CREATE TABLE Admin (  
  admin_id INT PRIMARY KEY,  
  user_id INT NOT NULL UNIQUE REFERENCES "User"(user_id) ON DELETE CASCADE  
);
```

-- 4. VENDOR

```
CREATE TABLE Vendor (  
  vendor_id INT PRIMARY KEY,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  phone VARCHAR(20),  
  bank_info TEXT,  
  rating NUMERIC(3,2) CHECK (rating BETWEEN 0 AND 5)  
);
```

-- 5. CATEGORY

```
CREATE TABLE Category (  
  category_id SERIAL PRIMARY KEY,  
  cat_name VARCHAR(100) NOT NULL,  
  parent_cat_id INT REFERENCES Category(category_id) ON DELETE SET NULL  
);
```

-- 6. PRODUCT


```
CREATE TABLE Product (
  product_id SERIAL PRIMARY KEY,
  p_name VARCHAR(200) NOT NULL,
  description TEXT,
  base_price NUMERIC(12,2) NOT NULL CHECK (base_price >= 0),
  current_price NUMERIC(12,2) NOT NULL CHECK (current_price >= 0),
  category_id INT REFERENCES Category(category_id) ON DELETE SET NULL,
  vendor_id INT NOT NULL REFERENCES Vendor(vendor_id) ON DELETE CASCADE
);
```

-- 7. WAREHOUSE

```
CREATE TABLE Warehouse (
  warehouse_id SERIAL PRIMARY KEY,
  w_name VARCHAR(150) NOT NULL,
  location VARCHAR(200),
  region VARCHAR(100)
);
```

-- 8. WAREHOUSE_MANAGER

```
CREATE TABLE Warehouse_Manager (
  mgr_id SERIAL PRIMARY KEY,
  mgr_email VARCHAR(150) NOT NULL UNIQUE,
  warehouse_id INT NOT NULL UNIQUE REFERENCES Warehouse(warehouse_id) ON DELETE CASCADE
);
```

-- 9. ORDERS

```
CREATE TABLE Orders (
  order_id SERIAL PRIMARY KEY,
  order_date DATE NOT NULL DEFAULT CURRENT_DATE,
  status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  total_amount NUMERIC(12,2) NOT NULL CHECK (total_amount >= 0),
  customer_id INT NOT NULL REFERENCES Customer(customer_id) ON DELETE RESTRICT
);
```

-- 10. ORDER_ITEM

```
CREATE TABLE Order_Item (
  order_id INT NOT NULL REFERENCES Orders(order_id) ON DELETE CASCADE,
  item_no INT NOT NULL,
  product_id INT NOT NULL REFERENCES Product(product_id) ON DELETE RESTRICT,
  quantity INT NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
  PRIMARY KEY (order_id, item_no)
);
```

-- 11. PAYMENT

```
CREATE TABLE Payment (
  payment_id SERIAL PRIMARY KEY,
  method VARCHAR(50) NOT NULL,
  status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  amount NUMERIC(12,2) NOT NULL CHECK (amount >= 0),
  paid_at TIMESTAMP,
  order_id INT NOT NULL UNIQUE REFERENCES Orders(order_id) ON DELETE CASCADE
);
```

-- 12. DELIVERY_AGENT

```
CREATE TABLE Delivery_Agent (
  agent_id SERIAL PRIMARY KEY,
  ag_name VARCHAR(100) NOT NULL,
  phone VARCHAR(20),
  vehicle_type VARCHAR(50),
  region VARCHAR(100)
);
```

-- 13. DELIVERY

```
CREATE TABLE Delivery (
  delivery_id SERIAL PRIMARY KEY,
  status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  est_time TIMESTAMP,
  actual_time TIMESTAMP,
  tracking_no VARCHAR(100) NOT NULL UNIQUE,
  order_id INT NOT NULL UNIQUE REFERENCES Orders(order_id) ON DELETE CASCADE,
  agent_id INT REFERENCES Delivery_Agent(agent_id) ON DELETE SET NULL
);
```

-- 14. REVIEW

```
CREATE TABLE Review (
  review_id SERIAL PRIMARY KEY,
  rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment TEXT,
  reviewed_on DATE NOT NULL DEFAULT CURRENT_DATE,
  customer_id INT NOT NULL REFERENCES Customer(customer_id) ON DELETE CASCADE,
  product_id INT NOT NULL REFERENCES Product(product_id) ON DELETE CASCADE
);
```

-- 15. RETURN_REQUEST

```
CREATE TABLE Return_Request (
  return_id SERIAL PRIMARY KEY,
  reason TEXT,
  status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  requested_on DATE NOT NULL DEFAULT CURRENT_DATE,
  order_id INT NOT NULL REFERENCES Orders(order_id) ON DELETE CASCADE
);
```

-- 16. REFUND

```
CREATE TABLE Refund (
  refund_id SERIAL PRIMARY KEY,
  amount NUMERIC(12,2) NOT NULL CHECK (amount >= 0),
  status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  processed_on DATE,
  return_id INT NOT NULL UNIQUE REFERENCES Return_Request(return_id) ON DELETE CASCADE
);
```

-- 17. COUPON

```
CREATE TABLE Coupon (
  coupon_id SERIAL PRIMARY KEY,
  code VARCHAR(50) NOT NULL UNIQUE,
  discount_pct NUMERIC(5,2) NOT NULL CHECK (discount_pct BETWEEN 0 AND 100),
  valid_from DATE NOT NULL,
  valid_to DATE NOT NULL,
  CHECK (valid_to >= valid_from)
);
```

-- 18. CAMPAIGN

```
CREATE TABLE Campaign (
  campaign_id SERIAL PRIMARY KEY,
  c_name VARCHAR(200) NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  target_segment VARCHAR(100),
  CHECK (end_date >= start_date)
);
```

-- 19. INVENTORY

```
CREATE TABLE Inventory (
  product_id INT NOT NULL REFERENCES Product(product_id) ON DELETE CASCADE,
  warehouse_id INT NOT NULL REFERENCES Warehouse(warehouse_id) ON DELETE CASCADE,
  stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
  PRIMARY KEY (product_id, warehouse_id)
);
```